

Reimagined Audio Player With Uploads

A modern, glassmorphic audio player for **Next.js**, **TypeScript**, **Tailwind CSS**, **shadcn/ui**, and **Framer Motion**. This version includes a full playlist queue plus drag-and-drop upload support for local `.mp3` and `.wav` files.

Features

- Drag-and-drop upload zone
- Browse-file upload fallback
- Supports `.mp3` and `.wav`
- Local preview playback with `URL.createObjectURL()`
- Uploaded files are added directly to the queue
- Remove uploaded tracks from the queue
- Built-in demo/library tracks
- Playlist queue
- Play, pause, previous, and next
- Shuffle and repeat modes
- Save/favorite state
- Download/export action
- Volume slider and mute toggle
- Accessible seek slider
- Keyboard support
- `Space` : play / pause
- `ArrowRight` : seek forward 10 seconds
- `ArrowLeft` : seek backward 10 seconds
- Animated cover transitions
- Motion waveform visualization
- Glassmorphism 2.0 interface

Tech Stack

- Next.js App Router
- React
- TypeScript
- Tailwind CSS
- shadcn/ui
- Radix UI primitives through shadcn/ui
- Framer Motion
- lucide-react

Installation

Install dependencies:

```
npm install framer-motion lucide-react
```

Install required shadcn/ui components:

```
npx shadcn@latest add button card badge slider
```

Recommended File Structure

```
app/  
  music/  
    page.tsx  
components/  
  ReimaginedAudioPlayerWithUploads.tsx  
public/  
  audio/  
    farm-dayzz.mp3  
  images/  
    farm-dayzz-cover.jpg
```

Add the Component

Save the component as:

```
components/ReimaginedAudioPlayerWithUploads.tsx
```

The component must be a client component because it uses browser APIs such as drag-and-drop, file input, audio playback, and `URL.createObjectURL()`.

```
"use client";
```

Basic Usage

```
import ReimaginedAudioPlayer from "@/components/  
ReimaginedAudioPlayerWithUploads";  
  
export default function MusicPage() {  
  return (  
    <main className="min-h-screen bg-background p-6">  
      <ReimaginedAudioPlayer />  
    </main>  
  );  
}
```

```

    </main>
  );
}

```

Usage With Custom Tracks

```

import ReimaginedAudioPlayer from "@/components/
ReimaginedAudioPlayerWithUploads";

const tracks = [
  {
    id: "farm-dayzz",
    title: "Farm Dayzz",
    artist: "Sterl",
    src: "/audio/farm-dayzz.mp3",
    cover: "/images/farm-dayzz-cover.jpg",
    mood: "Country Flow",
    durationHint: "--:--",
  },
  {
    id: "neon-drift",
    title: "Neon Drift",
    artist: "Sterl Audio Lab",
    src: "/audio/neon-drift.mp3",
    cover: "/images/neon-drift.jpg",
    mood: "Focus",
    durationHint: "3:18",
  },
];

export default function MusicPage() {
  return (
    <main className="min-h-screen bg-background p-6">
      <ReimaginedAudioPlayer
        tracks={tracks}
        initialTrackId="farm-dayzz"
        allowDownload
      />
    </main>
  );
}

```

Adding Static Audio Files

Place static audio files in the `public/audio` directory.

For **Farm Dayzz**, use:

```
public/audio/farm-dayzz.mp3
```

Reference it in the player with:

```
src: "/audio/farm-dayzz.mp3"
```

Files inside `public` are served from the site root, so `public/audio/farm-dayzz.mp3` becomes available at `/audio/farm-dayzz.mp3`.

Upload Behavior

The upload zone accepts:

```
.mp3  
.wav
```

Supported MIME checks include:

```
audio/mpeg  
audio/mp3  
audio/wav  
audio/x-wav
```

Uploaded files are not sent to a server. They are previewed locally in the browser using:

```
URL.createObjectURL(file)
```

When a local upload is removed, or when the component unmounts, the object URL is revoked with:

```
URL.revokeObjectURL(track.src)
```

This prevents unnecessary memory leaks.

Track Type

```
type Track = {  
  id: string;
```

```

title: string;
artist?: string;
src: string;
cover?: string;
mood?: string;
durationHint?: string;
local?: boolean;
};

```

Component Props

```

type AudioPlayerProps = {
  tracks?: Track[];
  initialTrackId?: string;
  allowDownload?: boolean;
  className?: string;
};

```

Prop	Type	Default	Description
<code>tracks</code>	<code>Track[]</code>	Demo tracks	Static playlist/library tracks.
<code>initialTrackId</code>	<code>string</code>	First track	Track to load first.
<code>allowDownload</code>	<code>boolean</code>	<code>true</code>	Enables download/export action.
<code>className</code>	<code>string</code>	<code>""</code>	Additional classes for the player wrapper.

Local Upload Flow

1. User drags an `.mp3` or `.wav` file into the upload zone.
2. Component validates the file type and extension.
3. A local object URL is created.
4. The uploaded file is added to the top of the queue.
5. The uploaded file can be played immediately.
6. User can remove the local upload from the queue.
7. The object URL is revoked to clean up memory.

Persisting Uploaded Songs

The current upload system is for local preview only. Refreshing the page clears uploaded files.

To persist uploaded audio, connect the upload flow to one of these storage options:

- Vercel Blob
- Supabase Storage

- AWS S3
- Cloudflare R2
- Firebase Storage

Recommended production flow:

1. Upload the file to storage.
2. Store metadata in your database.
3. Save fields such as `title`, `artist`, `src`, `cover`, `duration`, and `ownerId`.
4. Pass the saved track list into the player as the `tracks` prop.

Accessibility Notes

The player includes accessible labels for playback controls, `aria-pressed` states for toggle buttons, `aria-current` for the active queue item, keyboard seek support, and screen-reader-friendly error messaging through `role="alert"`.

The upload input is visually hidden with `sr-only`, while the visible upload card and browse button provide mouse, keyboard, and drag-and-drop access.

Browser Notes

- Autoplay may be blocked until the user interacts with the page.
- WAV files can be large and may take longer to load.
- Object URLs are local to the browser session.
- Uploaded files are not permanent unless you add a backend storage service.

Deployment

Deploy on Vercel for best compatibility with the Next.js App Router.

Before production deployment:

- Compress large audio files.
- Move static tracks into `public/audio` or a CDN.
- Use signed URLs for private audio.
- Add server-side storage for persistent uploads.
- Keep the audio player as a client-side island inside a server-rendered page shell.

UX Rationale

The upload zone is placed beside the playback controls so users can act on their intent immediately: listen, add, save, or export. Local uploads are inserted at the top of the queue to create instant feedback. The glassmorphic bento layout separates active playback, intent controls, and the queue while keeping the experience tactile, accessible, and fast.